

BLOC 1 - FRONT-END

Developpement Front-End de sites et applications web

Ce que le jury attendait sur chaque competence, et ce que j'ai fait pour y repondre.

Projet Airlines Reservation - Flask + PostgreSQL

Document de preparation orale au jury

C1 - HTML / CSS

C1.a Intégrer une maquette en HTML/CSS avec un code propre et structure

CE QUI ETAIT ATTENDU

Que le HTML soit valide et structure avec les vraies balises sémantiques (header, nav, main, footer), et que les composants répétés sur le site (cartes hotel, cartes vol) suivent une structure cohérente plutôt que d'être recopiés différemment sur chaque page.

CE QUE J'AI FAIT

J'ai unifié toutes les cartes hotel et vol du site (accueil, liste hotels, liste vols, recherche, reservation) autour d'un seul système de composants, avec une nomenclature de classes cohérente (.h-card pour les hotels, .v-card pour les vols). J'ai aussi corrigé un bug de balisage invalide présent sur 16 pages : le bouton de connexion était un `<button>` qui contenait un `<a>`, avec les balises fermées dans le mauvais ordre - ce qui casse la navigation au clavier et n'est pas du HTML valide. Je l'ai remplacé partout par un simple lien `<a>`.

-> static/css/styles.css, ligne 2239 : définition de .h-card

-> templates/index.html : `<a href="/logout" class="boutton_connect">`

C1.b Rendre le site responsive (mobile / tablette / desktop)

Point le plus critique du premier passage : note à 0/3, corrige intégralement

CE QUI ETAIT ATTENDU

Que le site s'affiche et fonctionne correctement sur mobile, tablette et desktop, sans débordement ni contenu tronqué, avec une navigation adaptée à chaque taille d'écran.

CE QUE J'AI FAIT

J'ai d'abord trouvé la cause racine du problème : une règle CSS globale (body en display:flex) empêchait la zone principale `<main>` de retrecir sous la largeur de son contenu le plus large. Dès qu'un élément large était présent quelque part sur une page - un tableau par exemple - toute la page débordait horizontalement et le texte se retrouvait tronqué sur mobile. Je l'ai identifié en injectant du JavaScript de débogage qui mesurait la largeur réelle des éléments, pas au hasard. Une fois cette règle corrigée, j'ai retravaillé un par un les éléments qui posaient problème : le formulaire de recherche de l'accueil (qui débordait), la barre de navigation (9 liens qui ne tenaient pas sur un écran mobile - j'ai gardé Accueil/Vols/Hotels visibles et cache le reste derrière une flèche), les tableaux larges de la page RGPD (rendus scrollables dans leur propre cadre au lieu de faire déborder toute la page), et les images (une version plus légère et recadrée est chargée sous 540px via la balise `<picture>`, au lieu de servir la même image lourde à tout le monde). J'ai vérifié chaque correction en 1920px, 768px et 375px avant de la considérer terminée.

-> static/css/styles.css, ligne 1335 : `body > main { flex: 1 1 auto; min-width: 0; width: 100%; }`

```
-> static/css/styles.css, ligne 488 : .search-bar-toggle
-> static/css/styles.css, ligne 351 : .nav-more-toggle
-> static/css/styles.css, ligne 2055 : .legal-table-wrap { overflow-x: auto; }
-> templates/index.html, ligne 111 : <picture><source media="(max-width: 540px)" ...>
```

C1.c Prendre en compte la diversité des publics (accessibilité, RGAA)

Corrige - police adaptée à la dyslexie ajoutée, plusieurs bugs d'accessibilité corrigés

CE QUI ETAIT ATTENDU

Que le site reste utilisable par des publics ayant des besoins spécifiques : troubles de lecture, navigation au clavier, lecteurs d'écran.

CE QUE J'AI FAIT

J'ai ajouté un bouton flottant, présent sur toutes les pages du site, qui permet de basculer vers une police adaptée à la dyslexie (OpenDyslexic). Le choix de l'utilisateur est mémorisé d'une visite à l'autre, et la police s'applique uniquement au texte - jamais aux icônes, qui utilisent leur propre police de glyphes et seraient rendues illisibles sinon. J'ai aussi corrigé le curseur "main" qui s'affichait sur toute une carte hotel ou vol alors que seul le bouton Reserver est réellement cliquable, ce qui induisait l'utilisateur en erreur. Enfin, la correction du bug de balisage invalide vue au C1.a a directement amélioré la navigation au clavier, puisqu'un lien mal fermé dans un bouton n'est pas correctement focusable.

```
-> static/assets/fonts/opendyslexic-regular.woff2 + opendyslexic-bold.woff2
-> static/css/styles.css, ligne 3272 : @font-face 'OpenDyslexic'
-> static/js/script.js, ligne 5 : initDyslexiaToggle()
```

C1.d Adopter une logique d'intégration réutilisable (système de classes)

CE QUI ETAIT ATTENDU

Que le CSS suive une organisation reconnue plutôt que des noms de classes improvisés au fil de l'eau.

CE QUE J'AI FAIT

J'ai suivi une nomenclature de type BEM (Block-Element-Modifier) pour tous les nouveaux composants, ce qui évite les conflits de noms et rend le CSS prévisible : par exemple .h-card__body, .v-card__footer, .admin-sidebar__nav. J'ai aussi rangé la feuille de style en sections clairement commentées et regroupées par fonctionnalité (page admin, dashboard, pages légales, accessibilité, carrousel...).

C1.e Soigner le référencement naturel (SEO)

CE QUI ETAIT ATTENDU

Que chaque page ait un titre unique, une hierarchie de titres logique, et n'affiche jamais de contenu vide ou casse.

CE QUE J'AI FAIT

Ce point etait deja largement en place. J'ai neanmoins trouve et corrige un vrai bug au passage : le titre des cartes vol s'affichait vide, car le code utilisait vol.nom, une colonne qui n'existe pas dans la table vols en base. Je l'ai remplace par vol.ville, qui est la donnee reellement disponible via la jointure avec la table destinations.

C2 - JavaScript

C2.a Enrichir l'interface avec du JavaScript (interactivite, animations)

CE QUI ETAIT ATTENDU

Que le site propose une interactivite reelle en JavaScript natif, sans se limiter a des formulaires HTML classiques.

CE QUE J'AI FAIT

J'ai developpe plusieurs composants interactifs en JavaScript natif, sans aucun framework JS type React ou Vue : un systeme de notifications (toast) generique reutilise partout (succes de formulaire, erreur de connexion, rate-limit depasse...), et plusieurs menus et bandeaux animes en JS pur - le menu du compte utilisateur, la navigation mobile repliable, les options de recherche repliables, et le bandeau de consentement cookies RGPD.

-> static/js/script.js, ligne 47 : navToggle - navigation mobile

-> static/js/script.js, ligne 59 : searchToggle - options de recherche

C2.b Valider les saisies utilisateur avec des methodes adaptees

CE QUI ETAIT ATTENDU

Que les formulaires filtrent et valident la saisie en temps reel, pas seulement au moment de l'envoi.

CE QUE J'AI FAIT

Sur le formulaire de paiement, les champs numero de carte, date d'expiration et CVV filtrent les caracteres invalides au fur et a mesure de la frappe et formatent automatiquement l'affichage (espace tous les 4 chiffres pour la carte, slash pour la date). Cote serveur, les mots de passe sont egalement verifies (8 caracteres minimum, confirmation identique) avant tout enregistrement en base, avec un message d'erreur explicite en retour.

-> templates/paiement.html : oninput="this.value = this.value.replace(/[^0-9]/g, ' ')"

C2.c Dialoguer avec le serveur sans recharger la page (requetes asynchrones)

Renforce avec de vrais verbes HTTP varies : PUT, PATCH, DELETE - pas seulement GET/POST

CE QUI ETAIT ATTENDU

Que le site utilise de vraies requetes asynchrones (fetch/AJAX) avec les verbes HTTP appropries a chaque action, pas uniquement GET et POST.

CE QUE J'AI FAIT

J'ai construit toute la gestion du compte utilisateur en AJAX : modification du profil (nom/prenom/email) en PUT, changement de mot de passe en PATCH - choisi deliberement plutot

que PUT, car PATCH represente une modification partielle d'une seule donnee alors que PUT remplace l'ensemble des champs - et suppression de compte en DELETE, avec confirmation par mot de passe. Le dashboard administrateur suit la meme logique pour gerer les comptes clients : creation en POST, modification en PUT, suppression en DELETE, le tout sans jamais recharger la page. Chaque requete transmet en plus le jeton CSRF via un en-tete HTTP (X-CSRFToken), necessaire puisque ces requetes envoient du JSON et non un formulaire classique.

```
-> templates/profil.html, ligne 200 : sendRequest('/profil', 'PUT', payload)
-> templates/profil.html, ligne 230 : sendRequest('/profil', 'DELETE', payload)
-> templates/admin_comptes.html, ligne 152 : apiRequest() - fonction generique fetch
-> templates/profil.html, ligne 184 : 'X-CSRFToken': CSRF_TOKEN
```

C2.d Optimiser le developpement avec une librairie JS externe

Corrige - avant : 0/3 (non acquis) au premier passage

CE QUI ETAIT ATTENDU

Qu'une librairie JavaScript externe soit integree et que son choix soit justifie, plutot que de tout recoder soi-meme.

CE QUE J'AI FAIT

J'ai integre Swiper.js (version 11.2.10), une librairie specialisee dans les carrousels tactiles, tres largement utilisee et documentee. Je l'ai choisie plutot que de recoder un carrousel moi-meme car gerer correctement le swipe tactile sur mobile, la navigation au clavier, la pagination et les fleches, en couvrant tous les cas limites (inertie du glissement, redimensionnement d'ecran), represente un travail consequent que Swiper fait de maniere fiable et deja testee. J'ai telecharge etheberge les fichiers en local dans le projet, plutot que de dependre d'un CDN externe qui pourrait etre indisponible le jour de l'examen. Je m'en sers concretement pour le carrousel "Nos coups de coeur" de la page d'accueil, avec une configuration adaptative : 1 slide visible sur mobile, jusqu'a 4 sur desktop.

```
-> static/vendor/swiper/swiper-bundle.min.js + swiper-bundle.min.css
-> templates/index.html, ligne 355 : new Swiper('.destinations-swiper', {...})
```

Bloc 1 - Front-End - Airlines Reservation. Document de preparation au jury de rattrapage.